

Prolog Program for Forward Chaining

Aim

To write a Prolog program to implement **Forward Chaining** and verify the derived facts using a query.

Algorithm

1. Start.
2. Define the initial facts.
3. Define rules for deriving new facts.
4. Select a rule whose condition matches an existing fact.
5. Add the derived fact to the knowledge base.
6. Repeat until no new facts can be derived.
7. Enter a query to check whether a fact is available.
8. Display the result.
9. Stop.

Prolog Program

```
% Forward Chaining

:- dynamic fact/1.

fact(sunny).
fact(warm).

rule(sunny, happy).
rule(warm, comfortable).
rule(happy, active).
rule(comfortable, relaxed).

forward :-
    fact(X),
    rule(X, Y),
    \+ fact(Y),
    assertz(fact(Y)),
    fail.

forward.

query(X) :-
    fact(X).
```

Query

```
?- forward.
```

Output:

```
true.
```

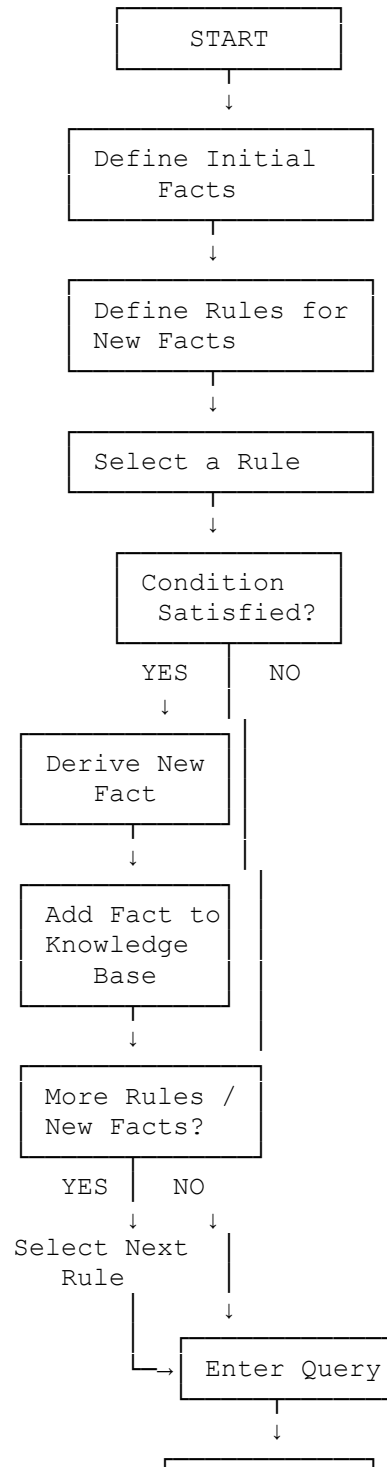
Then check the derived fact:

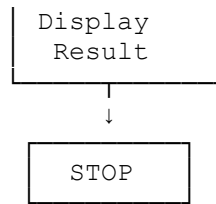
```
?- query(active).
```

Output:

```
true.
```

Flowchart





OUTPUT

```
SWI-Prolog (AMD64, Multi-threaded, version 9.2.9)
File Edit Settings Run Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.9)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?-
% c:/Users/Madhu Bavana/OneDrive/Desktop/Prl/Forward Chaining.pl compiled 0.00 sec, 9 clauses
?-
| forward.
true.

?- query(active).
false.

?-
```

Thus, Forward Chaining is successfully implemented in Prolog, and the derived fact is verified using a query.